CSE499A – Section 15 – Group 03
Md. Rahad Hossain, Md. Yousuf, MD. Rokib Hasan Oli

CityPulse: Intelligent Parking & Traffic Control Platform

Weekly Progress Report – Update 3 (Sprints 3–4 partial)

I. Sprint Overview

Sprint 3 targeted the identity and onboarding layer: JWT authentication with permission-based RBAC, refresh token rotation, multi-session support, and parking space registration with photo upload and RTSP camera connection. All Sprint 3 development tasks are complete. In parallel, the first two research objectives of Sprint 4 are also done: the object detection literature review and analysis of fixed-camera challenges for parking occupancy.

II. Sprint 3: Authentication

A. JWT with Permission-Based RBAC

Authentication uses JWT with permission-based access control. Permissions are resolved from the database at login and embedded as a `permissions[]` claim inside the token. A global guard checks these claims on every request with no per-request database lookup. Admin bypasses all permission checks. Adding access to a feature for a role requires a single row insertion in the `role_permissions` table, no code change.

B. Refresh Token Rotation and Multi-Session

Refresh tokens are stored as SHA-256 hashes in the `user_sessions` table, not in a separate simple table. Each session records the device name, IP address, user agent, and expiry. On refresh the old session is revoked and a new pair is issued (rotation). The `jti` claim in the access token maps to the session row, enabling per-session force-revoke from the authority dashboard. A user can hold concurrent sessions across multiple devices.

C. Auth Endpoints

Six endpoints are live under `/auth`:

- `POST /auth/register` — role-aware registration (driver, owner, authority) creating the user record and the corresponding profile row in one transaction.
- `POST /auth/login` — returns access token (15 min) and refresh token (30 days); records last login timestamp.
- `POST /auth/refresh` — verifies token signature and hash, revokes old session, issues new pair.
- `POST /auth/logout` — revokes the session identified by the `jti` claim.
- `GET /auth/sessions` — returns all active sessions for the calling user.
- `DELETE /auth/sessions/:id` — revokes a specific session by ID.

Route guards are applied globally via `APP_GUARD`. Public routes opt out with a `@Public()` decorator.

D. Role-Separated Registration

Registration accepts a `role` field and creates the matching profile automatically: `driver_profiles` (full name, FCM token), `owner_profiles` (business name, address, national ID), or `authority_profiles` (organization, badge number, jurisdiction). Passwords are hashed with bcrypt at 12 rounds.

III. Sprint 3: Parking Space Registration

A. Spaces API

A full parking space CRUD module is implemented at `/spaces`. Owners can create a space with name, coordinates, Bangladesh administrative zone fields (division, district, thana, area), capacity, space type, operating hours, amenities, base rate, and an optional RTSP stream URL. Up to ten photos may be uploaded per request (5 MB each) using multer disk storage. Photos are stored to `uploads/spaces/` with S3 key references in the `space_photos` table for a future migration to object storage. When an RTSP URL is provided at creation, a `cameras` record is created automatically with status `setup`.

TABLE I
Parking space endpoints

Endpoint	Action
`POST /spaces`	Create space + photos + camera
`GET /spaces`	List caller's spaces
`GET /spaces/:id`	Get one (owner or admin)
`PATCH /spaces/:id`	Update fields or RTSP URL
`DELETE /spaces/:id`	Soft delete
`POST /spaces/:id/photos`	Add more photos
`DELETE /spaces/:id/photos/:p`	Remove a photo

Ownership is enforced on every write: non-owners receive `403 Forbidden`. Soft delete sets `deleted_at` so historical booking data is preserved.

B. Web Frontend Auth

The Next.js frontend now has a complete authentication shell: landing page with role-based entry points, a signup page with role selection (driver, owner, authority), password strength checklist, and an auth-gated dashboard showing the logged-in user's role and email. The `AuthContext` provides login, signup, logout, session refresh, and password reset across the application via a cookie-backed repository pattern. Swapping to the real NestJS backend later requires only replacing the repository implementation — no page changes.

IV. Sprint 4 (Partial): CV Research & Model Training

A. Object Detection Approaches

A detailed comparison was conducted between YOLOv8 (object detection), background subtraction, and CNN classification for fixed-camera parking slot occupancy. The key finding is that for a fixed-camera single-lot setup, binary classification per slot patch outperforms full-scene detection

in both inference speed and accuracy on benchmark datasets. YOLOv8 requires annotating vehicles rather than slots, which is unnecessary overhead when slot boundaries are fixed. MobileNetV2 was selected as the classification backbone: it achieves competitive accuracy at a fraction of the compute cost, making real-time inference on low-cost edge hardware (Raspberry Pi 4 or Pi 5) feasible. Other papers building custom CNNs from scratch on the same dataset achieved around 93% accuracy, while pretrained MobileNet variants consistently reach 97–98%. The gap is entirely explained by transfer learning, not architecture differences.

B. Training Pipeline

Training is split into two phases, each in a separate Kaggle notebook. The dataset is a merged collection of PKLot (695,851 images from 3 Brazilian lots across sunny, overcast, and rainy conditions) and CNRPark+EXT (113,140 images from 9 Italian camera angles), totalling approximately 808,000 labelled images. Validation is performed on UFPR05, a completely unseen parking lot not present in the training split. This is a stricter evaluation than a random date-based split.

Phase 1 — Train the classifier head. All MobileNetV2 layers are frozen. A new 2-class output head (`Dropout(0.2)` followed by `Linear(1280, 2)`) is trained from scratch on top. Hyperparameters: batch size 128, learning rate 10^{-3}, Adam optimizer, 5 epochs, single GPU (Tesla T4).

TABLE II
PHASE 1 TRAINING RESULTS (VAL ON UFPR05)

Epoch	Train Loss	Train Acc	Val Loss	Val Acc
1	0.0466	0.9848	0.1094	0.9631
2	0.0421	0.9862	0.0916	0.9698
3	0.0423	0.9861	0.1057	0.9655
4	0.0414	0.9864	0.1178	0.9624
5	0.0416	0.9866	0.0803	**0.9728**

Best checkpoint: epoch 5, val accuracy **0.9728**. Weights are saved to Kaggle as a dataset (`mobilenetv2-parking-occupancy-phase1-weights`) for Phase 2 input.

Phase 2 — Full fine-tune (pending). Phase 1 weights are loaded, all layers are unlocked, and the whole network is trained at a much lower learning rate (10^{-4}) for 10 epochs. The low learning rate is required to avoid overwriting the useful features the backbone learned from ImageNet. Best checkpoint by validation accuracy is saved, not the final epoch.

C. Model Architecture

```
input (224x224 RGB)
  -> MobileNetV2 feature extractor
  -> AdaptiveAvgPool2d
  -> Dropout(0.2)
  -> Linear(1280, 2)
  -> Softmax -> argmax -> {empty, occupied}
```

Input images are normalized with ImageNet statistics (mean $[0.485, 0.456, 0.406]$, std $[0.229, 0.224, 0.225]$) and resized to 224×224.

V. AI Inference Service

The FastAPI inference service is operational on port 8001. It accepts a cropped slot image via `POST /infer`, runs it through the Phase 1 MobileNetV2 checkpoint at `weights/mobilenetv2/phase1_weights.pth`, and returns a label (`empty` or `occupied`) with a confidence score. Inference results are logged to MongoDB with slot ID, space ID, confidence, and timestamp. The service is protected by a shared API key. RTSP ingestion and real-time slot status push to NestJS are not yet implemented — those are the remaining Sprint 4 build tasks.

VI. Plan for Sprint 4 (Remaining)

Analyse fixed-camera CV challenges specific to Bangladeshi lots (lighting variance, occlusion, unmarked slots, RTSP instability) and document mitigation strategies. Run Phase 2 fine-tuning once local data is collected. Build the RTSP ingestion worker in the AI service using OpenCV with exponential backoff reconnect, and wire slot status updates to the NestJS backend so that `parking_slots.occupied` is updated in real time.

Artifacts

- Auth module source: https://github.com/rahad-dll/499/tree/main/applications/web/backend/src/auth
- Spaces module source: https://github.com/rahad-dll/499/tree/main/applications/web/backend/src/spaces
- FastAPI inference service: https://github.com/rahad-dll/499/tree/main/applications/ai/api-services
- MobileNetV2 training notebooks: https://github.com/rahad-dll/499/tree/main/applications/ai/models/parking-occupancy/mobilenetv2/notebooks
- Next.js frontend: https://github.com/rahad-dll/499/tree/main/applications/web/frontend